

Palindrome partitioning

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
class Solution {
```

```
public:
```

```
vector<vector<string>> partition(string s) {
```

```
    // Resultant vector to store all partitions
```

```
    vector<vector<string>> res;
```

```
    // Temporary vector to store current partition
```

```
    vector<string> path;
```

```
    // Start the depth-first search from index 0
```

```
    dfs(0, s, path, res);
```

```
    return res;
```

```
}
```

```
void dfs(int index, string s, vector<string> &path,  
vector<vector<string>> &res) {
```

```
    // If the index reaches the end of the string
```

```
    if (index == s.size()) {
```

```
        // Add the current partition to the result
```

```
        res.push_back(path);
```

```

        return;
    }

    // Iterate over the substring starting from
    'index'

    for (int i = index; i < s.size(); ++i) {

        // Check if the substring s[index..i] is a
        palindrome

        if (isPalindrome(s, index, i)) {

            // If true, add it to the current path

            path.push_back(s.substr(index, i - index +
1));

            // Recur for the remaining substring

            dfs(i + 1, s, path, res);

            // Backtrack: remove the last added
            substring

            path.pop_back();

        }

    }

}

```

```

bool isPalindrome(string s, int start, int end) {

    // Check if the substring s[start..end] is a
    palindrome

```

```
while (start <= end) {  
    // If characters do not match, it's not a  
    palindrome  
    if (s[start++] != s[end--])  
        return false;  
}  
return true; // Otherwise, it's a palindrome  
};
```

```
// Main method for testing  
int main() {  
    Solution solution;  
    string s = "aab";  
    vector<vector<string>> result =  
    solution.partition(s);  
    for (const auto& partition : result) {  
        for (const auto& str : partition) {  
            cout << str << " ";  
        }  
        cout << endl;  
    }  
}
```

```
    return 0;
}
```

Word Search

```
#include<bits/stdc++.h>

using namespace std;
```

```
class Solution {
public:
```

```
    // Helper function to check if the word exists
    starting from cell (i, j)
```

```
    bool func(vector<vector<char> > & v, int i, int j,
string& s, int k) {
```

```
        // If all characters of the word are found
```

```
        if (k == s.length()) {
```

```
            return true;
```

```
        }
```

```
        // Boundary conditions and character
        mismatch check
```

```
        if (i < 0 || j < 0 || i >= v.size() || j >= v[0].size() ||
s[k] != v[i][j]) {
```

```
            return false;
```

```
        }
```

```
// Initialize answer as false

bool ans = false;


// Temporarily mark the cell as visited

char x = v[i][j];

v[i][j] = ' ';


// Check all four possible directions (down, up,
right, left)

ans |= func(v, i + 1, j, s, k + 1);
ans |= func(v, i - 1, j, s, k + 1);
ans |= func(v, i, j + 1, s, k + 1);
ans |= func(v, i, j - 1, s, k + 1);


// Restore the original character in the cell

v[i][j] = x;


return ans;

}
```

// Main function to check if the word exists in the board

bool exist(vector<vector<char> >& board, string word) {

// Iterate through each cell in the board

for (int i = 0; i < board.size(); i++) {

for (int j = 0; j < board[0].size(); j++) {

// If the first character matches, start the search

if (board[i][j] == word[0]) {

// If the word is found, return true

if (func(board, i, j, word, 0)) {

return true;

}

}

}

}

// If the word is not found, return false

return false;

}

};

// Main function to test the solution

int main() {

Solution sol;

vector<vector<char> > board = {

{'A', 'B', 'C', 'E'},

{'S', 'F', 'C', 'S'},

{'A', 'D', 'E', 'E'}

};

string word = "ABCCED";

if (sol.exist(board, word)) {

cout << "Word found!" << endl;

} else {

cout << "Word not found!" << endl;

}

return 0;

}

N Queen

#include<bits/stdc++.h>

using namespace std;

```
class Solution {
```

```
public:
```

```
    // Check if it's safe to place a queen at  
board[row][col]
```

```
    bool isSafe(vector<string>& board, int row, int  
col) {
```

```
        int r = row, c = col;
```

```
        // Check for upper left diagonal
```

```
        while (r >= 0 && c >= 0) {
```

```
            if (board[r][c] == 'Q') return false;
```

```
            r--;
```

```
            c--;
```

```
        }
```

```
        // Reset to the original position
```

```
        r = row;
```

```
        c = col;
```

```
        // Check for top
```

```
        while (r >= 0) {
```



```
    if (board[r][c] == 'Q') return false;
    r--;
}
```

```
// Reset to the original position
```

```
r = row;
```

```
c = col;
```

```
// Check for top right diagonal
```

```
while (r >= 0 && c < board[0].size()) {
```

```
    if (board[r][c] == 'Q') return false;
```

```
    r--;
```

```
    c++;
```

```
}
```

```
// If no queens are found, it's safe
```

```
return true;
```

```
}
```

```
// Function to place queens on the board
```

```
void func(int row, vector<vector<string>>& ans,
vector<string>& board) {
```

// If all columns are filled, add the solution to the answer

```
if (row == board.size()) {  
    ans.push_back(board);  
    return;  
}
```

// Try placing a queen in each column for the current row

```
for (int col = 0; col < board[0].size(); col++) {  
    // Check if it's safe to place a queen  
    if (isSafe(board, row, col)) {  
        // Place the queen  
        board[row][col] = 'Q';
```

// Recursively place queens in the next rows

```
func(row + 1, ans, board);
```

// Remove the queen and backtrack

```
board[row][col] = '.';  
}
```

```
    }  
}
```

// Solve the N-Queens problem

```
vector<vector<string>> solveNQueens(int n) {
```

```
    // List to store the solutions
```

```
    vector<vector<string>> ans;
```

```
    // Initialize the board with empty cells
```

```
    vector<string> board(n, string(n, '.'));
```

```
    // Start placing queens from the first column
```

```
    func(0, ans, board);
```

```
    return ans;
```

```
}
```

```
};
```

// Main method to test the solution

```
int main() {
```

```
    Solution solution;
```

```
    int n = 4; // Example with 4 queens
```

```
    vector<vector<string>> solutions =  
    solution.solveNQueens(n);
```

```

// Print all solutions

for (const auto& sol : solutions) {
    for (const auto& row : sol) {
        cout << row << endl;
    }
    cout << endl;
}

return 0;
}

```

Rat in a Maze

```

#include<bits/stdc++.h>

```

```

using namespace std;

```

```

// Vector to store the resulting paths

```

```

vector<string> result;

```

```

class Solution {

```

```

public:

```

```

    // Recursive function to find paths

```

```
void path(vector<vector<int>>& m, int x, int y,  
string dir, int n) {  
    // If destination is reached, add path to result  
    if (x == n - 1 && y == n - 1) {  
        result.push_back(dir);  
        return;  
    }  
  
    // If cell is blocked, return  
    if (m[x][y] == 0) return;  
  
    // Mark cell as visited by setting it to 0  
    m[x][y] = 0;  
  
    // Move up if possible  
    if (x > 0) path(m, x - 1, y, dir + 'U', n);  
    // Move left if possible  
    if (y > 0) path(m, x, y - 1, dir + 'L', n);  
    // Move down if possible  
    if (x < n - 1) path(m, x + 1, y, dir + 'D', n);  
    // Move right if possible  
    if (y < n - 1) path(m, x, y + 1, dir + 'R', n);
```

```

// Unmark cell as visited by setting it to 1

m[x][y] = 1;

}


// Function to find all paths

vector<string> findPath(vector<vector<int>>&
grid) {

    int n = grid.size();


    // Clear previous results

    result.clear();


    // If starting or ending cell is blocked, return
empty result

    if (grid[0][0] == 0 || grid[n - 1][n - 1] == 0) return
result;


    // Start finding paths from (0, 0)

    path(grid, 0, 0, "", n);


    // Sort the result paths

```

```
sort(result.begin(), result.end());

return result;
}
};


int main() {

// Define the grid

vector<vector<int>> grid = {
    {1, 0, 0, 0, 0},
    {1, 1, 0, 1, 1},
    {0, 1, 0, 0, 1},
    {0, 1, 1, 1, 1},
    {0, 0, 0, 0, 1}
};


// Create an instance of the Solution class

Solution sol;


// Find all paths in the grid

vector<string> paths = sol.findPath(grid);
```

```

// Print all the found paths
for (const string& path : paths) {
    cout << path << endl;
}

return 0;
}

```

M Coloring Problem

```

#include <bits/stdc++.h>
using namespace std;

class Solution {
public:

    // Function to check if it's safe to color the node
with a given color

    bool isSafe(int col, int node, vector<int>& colors,
vector<int> adj[]) {

        // Check adjacent nodes

        for (int i = 0; i < adj[node].size(); i++) {

            // If an adjacent node has the same color

```



```
        if (colors[adj[node][i]] == col) return false;
    }
    return true; // Safe to color
}
```

// Recursive function to solve graph coloring problem

```
bool solve(int node, int m, int n, vector<int>&
colors, vector<int> adj[]) {
    // If all nodes are colored
    if (n == node) return true;
    // Try all colors from 1 to m
    for (int i = 1; i <= m; i++) {
        // Check if it is safe to color the node with
        color i
        if (isSafe(i, node, colors, adj)) {
            // Assign color i to node
            colors[node] = i;
            // Recursively try to color the next node
            if (solve(node + 1, m, n, colors, adj))
                return true;
            // Reset color if it doesn't lead to a
            solution
        }
    }
}
```

```
        colors[node] = 0;
    }
}

// No color can be assigned
return false;
}
```

// Function to check if the graph can be colored with m colors

```
bool graphColoring(vector<vector<int>>& edges,  
int m, int n) {
```

```
// Adjacency list representation of the graph  
vector<int> adj[n];
```

```
// Build the graph from edges
```

```
for (int i = 0; i < edges.size(); i++) {  
    adj[edges[i][0]].push_back(edges[i][1]);  
    adj[edges[i][1]].push_back(edges[i][0]);  
}
```

```
// Initialize all colors to 0 (uncolored)
```

```
vector<int> colors(n, 0);
```

```
// Start solving from the first node
```

```

        return solve(0, m, n, colors, adj);
    }
};

int main() {
    Solution sol;
    vector<vector<int>> edges = {
        {0, 1}, {0, 2}, {1, 2}, {1, 3}
    };
    int m = 3; // Number of colors
    int n = 4; // Number of nodes

    // Check if the graph can be colored with m
    colors
    if (sol.graphColoring(edges, m, n)) {
        cout << "The graph can be colored with " << m
        << " colors." << endl;
    } else {
        cout << "The graph cannot be colored with "
        << m << " colors." << endl;
    }
}

```

```

    return 0;
}

Sudoku Solver

#include<bits/stdc++.h>

using namespace std;

class Solution {
public:
    void solveSudoku(vector<vector<char>>& board)
    {
        solve(board);
    }

private:
    // Recursive method to solve the Sudoku
    bool solve(vector<vector<char>>& board) {
        // Size of the board
        int n = 9;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                // Empty cell found
                if (board[i][j] == '.') {

```

```

        for (char digit = '1'; digit <= '9'; digit++)
    {
        // Check if digit can be placed
        if (areRulesMet(board, i, j, digit)) {
            // Place digit
            board[i][j] = digit;

            // Recur to place next digits
            if (solve(board)) {
                return true;
            } else {
                // Reset if placing digit doesn't
                solve Sudoku
                board[i][j] = '.';
            }
        }

        // If no digit can be placed, return false
        return false;
    }
}

return true; // Sudoku solved

```

```
}
```

**// Method to check if placing a digit follows
Sudoku rules**

```
bool areRulesMet(vector<vector<char>>& board,  
int row, int col, char digit) {
```

```
    for (int i = 0; i < 9; i++) {
```

```
        if (board[row][i] == digit || board[i][col] ==  
digit) {
```

```
            // Digit already in row or column
```

```
            return false;
```

```
        }
```

```
    }
```

```
    int startRow = (row / 3) * 3;
```

```
    int startCol = (col / 3) * 3;
```

```
    for (int i = startRow; i < startRow + 3; i++) {
```

```
        for (int j = startCol; j < startCol + 3; j++) {
```

```
            if (board[i][j] == digit) {
```

```
                // Digit already in 3x3 sub-box
```

```
                return false;
```

```
            }
```

```
    }
```

```

    }

    // Digit can be placed

    return true;

}

};

int main() {

    Solution solution;

    vector<vector<char>> board = {
        {'5', '3', '.', '.', '7', '.', '.', '.', '.'},
        {'6', '.', '.', '1', '9', '5', '.', '.', '.'},
        {'.', '9', '8', '.', '.', '.', '.', '6', '.'},
        {'8', '.', '.', '.', '6', '.', '.', '.', '3'},
        {'4', '.', '.', '8', '.', '3', '.', '.', '1'},
        {'7', '.', '.', '.', '2', '.', '.', '.', '6'},
        {'.', '6', '.', '.', '.', '.', '2', '8', '.'},
        {'.', '.', '.', '4', '1', '9', '.', '.', '5'},
        {'.', '.', '.', '.', '8', '.', '.', '7', '9'}
    };

    solution.solveSudoku(board);

    for (const auto& row : board) {

```

```

        for (char c : row) {
            cout << c << " ";
        }
        cout << endl;
    }
    return 0;
}

```

Traversal in Linked List

```

#include <bits/stdc++.h>
using namespace std;

```

// Definition of singly linked list

```

class ListNode{
public:
    int data;
    ListNode *next;
    ListNode() : data(0), next(nullptr) {}
    ListNode(int x) : data(x), next(nullptr) {}
    ListNode(int x, ListNode *next) : data(x),
next(next) {}
};

```



```
class Solution {  
public:  
  
    //Function for Linked List Traversal  
vector<int> LLTraversal(ListNode* head) {  
    //Storing a copy of the linked list  
  
    ListNode* temp = head;  
  
    //To store the values  
  
    //Sequentially  
  
    vector<int> ans;  
  
  
    //Keep traversing  
  
    //Until the nullptr  
  
    //Is not encountered  
  
    while (temp != nullptr) {  
        //Storing of the values  
  
        ans.push_back(temp->data);  
  
        //Storing the address of the next node  
  
        temp = temp->next;  
  
    }  
  
    //Return answer  
  
    return ans;
```

```
    }  
};
```

```
int main() {
```

```
    //Manual creation of nodes
```

```
    ListNode* y1 = new ListNode(2);
```

```
    ListNode* y2 = new ListNode(5);
```

```
    ListNode* y3 = new ListNode(8);
```

```
    ListNode* y4 = new ListNode(7);
```

```
    // Linking the nodes
```

```
    y1->next = y2;
```

```
    y2->next = y3;
```

```
    y3->next = y4;
```

```
    //Instance of
```

```
    //Solution class
```

```
    Solution sol;
```

```
    //Calling LLTraversal method
```

```
    //To get the values
```

```
vector<int> result = sol.LLTraversal(y1);

//Printing the result

cout << "Linked List Values:" << endl;
for (int val : result) {
    cout << val << " ";
}
cout << endl;

//Clean up
//Allocated memory
delete y1;
delete y2;
delete y3;
delete y4;

return 0;
}
```

Deletion of the head of LL

```
#include <bits/stdc++.h>
using namespace std;
```

// Node structure

struct ListNode {

int val;

ListNode *next;

ListNode(): val(0), next(nullptr) {}

ListNode(int data1): val(data1), next(nullptr) {}

**ListNode(int data1, ListNode *next1): val(data1),
next(next1) {}**

};

class Solution {

public:

**// Function to delete the head node of the linked
list**

ListNode* deleteHead(ListNode* head) {

// If list is empty, nothing to delete

if (head == nullptr)

return nullptr;

// Set temporary pointer

ListNode* temp = head;

```
// Update head to the next node  
head = head->next;  
  
// Delete original head  
delete temp;  
  
// Return new head  
return head;  
}  
};
```

```
// Function to print the linked list  
void printList(ListNode* head) {  
    ListNode* current = head;  
    while (current != nullptr) {  
        cout << current->val << " ";  
        current = current->next;  
    }  
    cout << endl;  
}
```

// Function to insert a new node at the beginning of the linked list

```
ListNode* insertAtHead(ListNode* head, int data) {  
    ListNode* newNode = new ListNode(data);  
    newNode->next = head;  
    head = newNode;  
    return head;  
}
```

int main() {

// Create a linked list

ListNode* head = nullptr;

head = insertAtHead(head, 3);

head = insertAtHead(head, 2);

head = insertAtHead(head, 1);

cout << "Original list: ";

printList(head);

// Creating an instance of Solution Class

Solution sol;

```

// Function call to delete the head node

head = sol.deleteHead(head);


cout << "List after deleting head: ";
printList(head);


return 0;
}

```

Deletion of the tail of Linked List

```

#include <bits/stdc++.h>

using namespace std;


// Node structure

struct ListNode {
    int val;
    ListNode *next;
    ListNode(): val(0), next(nullptr) {}
    ListNode(int data1): val(data1), next(nullptr) {}
    ListNode(int data1, ListNode *next1): val(data1),
next(next1) {}
};

```

```
class Solution {
```

```
public:
```

```
// Function to delete the tail node of linked list
```

```
ListNode* deleteTail(ListNode* head) {
```

```
// If the list is empty or has only one node
```

```
if (head == NULL || head->next == NULL)
```

```
    return NULL; // Return NULL
```

```
// Temporary pointer
```

```
ListNode* temp = head;
```

```
/*Traverse to the second last
```

```
node in the list*/
```

```
while (temp->next->next != NULL) {
```

```
    temp = temp->next;
```

```
}
```

```
// Delete the last node
```

```
delete temp->next;
```



```
/*Set the next of the second  
last node to nullptr,  
effectively removing the last node*/  
temp->next = nullptr;  
  
// Return head of modified list  
return head;  
}  
};
```

```
// Function to print the linked list  
void printList(ListNode* head) {  
    ListNode* current = head;  
    while (current != nullptr) {  
        cout << current->val << " ";  
        current = current->next;  
    }  
    cout << endl;  
}
```

// Function to insert a new node at the beginning of the linked list

```
ListNode* insertAtHead(ListNode* head, int data) {  
    ListNode* newNode = new ListNode(data);  
    newNode->next = head;  
    head = newNode;  
    return head;  
}
```

int main() {

// Create a linked list

ListNode* head = nullptr;

head = insertAtHead(head, 3);

head = insertAtHead(head, 2);

head = insertAtHead(head, 1);

cout << "Original list: ";

printList(head);

// Creating an instance of Solution Class

Solution sol;

```

// Function call to delete the tail node

head = sol.deleteTail(head);

cout << "List after deleting head: ";
printList(head);

return 0;
}

```

Deletion of the Kth element of Linked List

```

#include <bits/stdc++.h>

using namespace std;

// Node structure

struct ListNode {
    int val;
    ListNode *next;
    ListNode(): val(0), next(nullptr) {}
    ListNode(int data1): val(data1), next(nullptr) {}
    ListNode(int data1, ListNode *next1): val(data1),
next(next1) {}
};

```

```
class Solution {  
public:  
  
    // Function to delete the k-th node of a linked list  
    ListNode* deleteKthNode(ListNode* head, int k)  
{  
  
    // If the list is empty, return NULL  
    if (head == NULL)  
        return NULL;  
  
    // If k is 1, delete the head node  
    if (k == 1) {  
        ListNode* temp = head;  
        head = head->next;  
        delete temp;  
        return head;  
    }  
  
    // Initialize a temporary pointer  
    ListNode* temp = head;  
  
    // Traverse to the (k-1)th node  
    for (int i = 0; temp != NULL && i < k - 2; i++) {
```

```

        temp = temp->next;
    }

    /*If k is greater than the number of nodes,
    return the unchanged list*/
    if (temp == NULL || temp->next == NULL)
        return head;

    // Delete the k-th node
    ListNode* next = temp->next->next;
    delete temp->next;
    temp->next = next;

    // Return head
    return head;
}

};

// Function to print the linked list
void printLL(ListNode* head) {
    ListNode* current = head;

```

```
while (current != nullptr) {  
    cout << current->val << " ";  
    current = current->next;  
}  
cout << endl;  
}  
  
int main() {  
    // Initialize a vector with values for the linked list  
    vector<int> arr = {12, 5, 8, 7};  
  
    // Create a linked list with the values from the  
vector  
    ListNode* head = new ListNode(arr[0]);  
    head->next = new ListNode(arr[1]);  
    head->next->next = new ListNode(arr[2]);  
    head->next->next->next = new ListNode(arr[3]);  
  
    // Print the original linked list  
    cout << "Original list: ";  
    printLL(head);
```

```
// Creating an instance of Solution class  
Solution sol;  
  
// Call the deleteKthNode function to delete the  
k-th node  
  
int k = 2;  
  
head = sol.deleteKthNode(head, k);  
  
// Print the linked list after deletion  
  
cout << "List after deleting the kth node: ";  
  
printLL(head);  
  
return 0;  
  
}
```

Delete the element with value X

```
#include <bits/stdc++.h>  
  
using namespace std;
```

// Definition of singly linked list

```
class ListNode{  
  
public:  
  
int data;
```

```
ListNode *next;  
ListNode() : data(0), next(nullptr) {}  
ListNode(int x) : data(x), next(nullptr) {}  
ListNode(int x, ListNode *next) : data(x),  
next(next) {}  
};
```

// Solution class

```
class Solution {
```

```
public:
```

```
// To delete a node with a specific value in a  
linked list
```

```
ListNode* deleteNodeWithValueX(ListNode*  
&head, int X) {
```

```
// Check if list is empty
```

```
if (head == NULL)
```

```
    return head;
```

```
// If first node has target value, delete
```

```
if (head->data == X) {
```

```
    ListNode* temp = head;
```

```
    head = head->next;
```



```
delete temp;  
return head;  
}
```

```
ListNode* temp = head;  
ListNode* prev = NULL;
```

```
/*Traverse the list to find  
the node with the target value*/  
while (temp != NULL) {  
    if (temp->data == X) {  
        // Adjust the pointers  
        prev->next = temp->next;  
        // Delete node  
        delete temp;  
        return head;  
    }  
    prev = temp;  
    temp = temp->next;  
}
```

```
        return head;
    }
};
```

// Helper Function to print the linked list

```
void printLL(ListNode* head) {
    while (head != NULL) {
        cout << head->data << " ";
        head = head->next;
    }
    cout << endl;
}
```

```
int main() {
    // Create a linked list from a vector
    vector<int> arr = {0, 1, 2};
    int X = 1;
    ListNode* head = new ListNode(arr[0]);
    head->next = new ListNode(arr[1]);
```

```
head->next->next = new ListNode(arr[2]);
```

```
// Print the original list
```

```
cout << "Original List: ";
```

```
printLL(head);
```

```
// Create a Solution object
```

```
Solution sol;
```

```
head = sol.deleteNodeWithValueX(head, X);
```

```
// Print the modified linked list
```

```
cout << "List after deleting the given value: ";
```

```
printLL(head);
```

```
return 0;
```

```
}
```

Insertion at the head of Linked List

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// Definition of singly linked list
```

struct ListNode

{

int val;

ListNode *next;

ListNode(int data1)

{

val = data1;

next = NULL;

}

ListNode(int data1, ListNode *next1)

{

val = data1;

next = next1;

}

};

// Solution class

class Solution {

public:

// Function to insert at head

ListNode* insertAtHead(ListNode* head, int X) {

```
//Creating a new node  
ListNode* newnode = new ListNode(X);  
  
/*Making next of newly created node to  
point the head of the LinkedList*/  
newnode->next = head;  
  
// Making newly created node as head  
head = newnode;  
  
// Return the head of modified list  
return head;  
}  
};
```

```
// Helper Function to print the linked list  
void printLL(ListNode* head) {  
    while (head != NULL) {  
        cout << head->val << " ";  
        head = head->next;  
    }  
}
```

```
}  
cout << endl;  
}
```

```
int main() {  
  
// Create a linked list from a vector  
vector<int> arr = {20, 30, 40};  
  
int X= 10;  
  
ListNode* head = new ListNode(arr[0]);  
head->next = new ListNode(arr[1]);  
head->next->next = new ListNode(arr[2]);  
  
  
// Print the original list  
cout << "Original List: ";  
printLL(head);  
  
  
// Create a Solution object  
  
Solution sol;  
  
head = sol.insertAtHead(head, X);  
  
  
// Print the modified linked list
```

```
cout << "List after inserting the given value at  
head: ";
```

```
printLL(head);
```

```
return 0;
```

```
}
```

Insertion at the tail of Linked List

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// Definition of singly linked list
```

```
struct ListNode
```

```
{
```

```
int val;
```

```
ListNode *next;
```

```
ListNode(int data1)
```

```
{
```

```
val = data1;
```

```
next = NULL;
```

```
}
```

```
ListNode(int data1, ListNode *next1)
```

```
{
```

```
    val = data1;
    next = next1;
}
};
```

// Solution class

```
class Solution {
```

```
public:
```

```
    // Function to insert a new node at the tail of the  
linked list
```

```
    ListNode* insertAtTail(ListNode* &head, int X) {
```

```
        if (head == NULL)
```

```
            return new ListNode(X);
```

```
        ListNode* temp = head;
```

```
        // Traversing until the last node
```

```
        while (temp->next != NULL) {
```

```
            temp = temp->next;
```

```
        }
```

```
        ListNode* newNode = new ListNode(X);
```

```
        temp->next = newNode;
```



```
        return head;
    }
};
```

// Helper Function to print the linked list

```
void printLL(ListNode* head) {
    while (head != NULL) {
        cout << head->val << " ";
        head = head->next;
    }
    cout << endl;
}
```

```
int main() {
    // Create a linked list from a vector
    vector<int> arr = {10, 20, 30};
    int val = 40;
    ListNode* head = new ListNode(arr[0]);
    head->next = new ListNode(arr[1]);
    head->next->next = new ListNode(arr[2]);
}
```

```

// Print the original list

cout << "Original List: ";
printLL(head);


// Create a Solution object
Solution sol;
head = sol.insertAtTail(head, val);


// Print the modified linked list

cout << "List after inserting the given value at
the tail:";
printLL(head);


return 0;
}

```

Insertion at the Kth position of Linked List

```

#include <bits/stdc++.h>
using namespace std;


// Definition of singly linked list
struct ListNode

```

```

{
    int val;
    ListNode *next;
    ListNode(int data1)
    {
        val = data1;
        next = NULL;
    }
    ListNode(int data1, ListNode *next1)
    {
        val = data1;
        next = next1;
    }
};

```

// Solution class

class Solution {

public:

// Function to insert a new node at the kth position

ListNode* insertAtKthPosition(ListNode* &head, int X, int K) {

```
/* If the linked list is empty  
and k is 1, insert the  
new node as the head*/  
if (head == NULL) {  
    if (K == 1)  
        return new ListNode(X);  
    else  
        return head;  
}
```

```
/*If K is 1, insert the new  
node at the beginning  
of the linked list*/  
if (K == 1)  
    return new ListNode(X, head);
```

```
int cnt = 0;  
ListNode* temp = head;
```

```
/* Traverse the linked list  
to find the node at position k-1*/
```

```
while (temp != NULL) {  
    cnt++;  
    if (cnt == K-1) {  
        /* Insert the new node after the node  
        at position k-1 */  
        ListNode* newNode = new ListNode(X,  
temp->next);  
        temp->next = newNode;  
        break;  
    }  
    temp = temp->next;  
}  
  
return head;  
}  
};
```

// Helper Function to print the linked list

```
void printLL(ListNode* head) {  
    while (head != NULL) {  
        cout << head->val << " ";  
        head = head->next;  
    }
```

```
    }  
    cout << endl;  
}
```

```
int main() {  
    // Create a linked list from a vector  
    vector<int> arr = {10, 30, 40};  
    int X = 20, K = 2;  
    ListNode* head = new ListNode(arr[0]);  
    head->next = new ListNode(arr[1]);  
    head->next->next = new ListNode(arr[2]);  
  
    // Print the original list  
    cout << "Original List: ";  
    printLL(head);  
  
    // Create a Solution object  
    Solution sol;  
    head = sol.insertAtKthPosition(head, X, K);  
  
    // Print the modified linked list
```

```
cout << "List after inserting the given value at  
the Kth position: ";
```

```
printLL(head);
```

```
return 0;
```

```
}
```

Insertion before the value X in Linked List

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// ListNode definition
```

```
class ListNode{
```

```
public:
```

```
int data;
```

```
ListNode *next;
```

```
ListNode() : data(0), next(nullptr) {}
```

```
ListNode(int x) : data(x), next(nullptr) {}
```

```
ListNode(int x, ListNode *next) : data(x),  
next(next) {}
```

```
};
```

```
// Solution class
```

```
class Solution {  
  
public:  
  
    // Function to insert a new node before the given  
node  
  
    ListNode* insertBeforeX(ListNode* &head, int X,  
int val) {  
  
        if (head == NULL) {  
            return NULL;  
        }  
  
        /* Insert at the beginning if the  
value matches the head's data */  
        if (head->data == X)  
            return new ListNode(val, head);  
  
        ListNode* temp = head;  
        while (temp->next != NULL) {  
            /* Insert at the current position if the  
next node has the desired value */  
            if (temp->next->data == X) {  
                ListNode* newNode = new ListNode(val,  
temp->next);
```



```
        temp->next = newNode;
        break;
    }
    temp = temp->next;
}
return head;
}
};
```

// Helper Function to print the linked list

```
void printLL(ListNode* head) {
    while (head != NULL) {
        cout << head->data << " ";
        head = head->next;
    }
    cout << endl;
}
```

```
int main() {
```

// Create a linked list from a vector

```
vector<int> arr = {1, 2, 4, 5};
```

```
int X = 4, val = 3;  
ListNode* head = new ListNode(arr[0]);  
head->next = new ListNode(arr[1]);  
head->next->next = new ListNode(arr[2]);  
  
// Print the original list  
cout << "Original List: ";  
printLL(head);  
  
// Create a Solution object  
Solution sol;  
head = sol.insertBeforeX(head, X, val);  
  
// Print the modified linked list  
cout << "List after inserting a new node before  
the given node: ";  
printLL(head);  
  
return 0;  
}
```